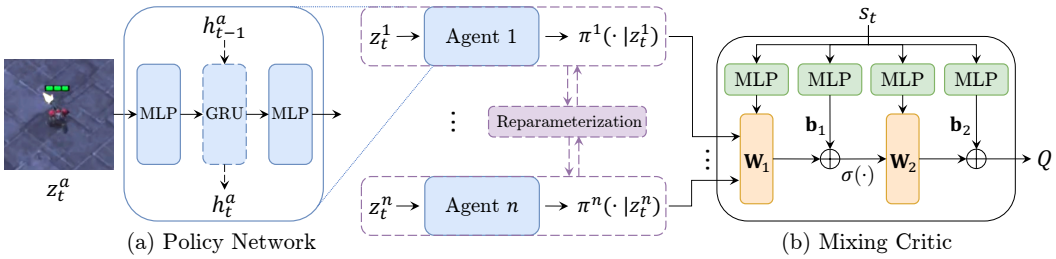

(a) Policy Network (b) Mixing Critic

Figure 1: **Overview of LICA**. (a) Architecture for the decentralized policy networks. (b) Architecture for the centralized mixing critic network. Dashed components are situational. Best viewed in color.

To this end, a simple approach taken by MADDPG [27] is to formulate Q^π_ϕ as an MLP that maps the concatenation of the state and actions (or more generally, their learned representations) into the joint Q estimate. In LICA, we instead propose to formulate Q^π_ϕ as a *hypernetwork* [11] that maps the state into a set of weights which, in turn, maps the concatenated action vectors into the Q estimate. We refer the resulting critic as the *mixing critic*, which is illustrated in Fig. 1. The key insight of this structural design is that vector concatenation in an MLP critic implies that the representations of the state s and the joint action u are associated through *addition*; with a critic hypernetwork, the additive association is converted to a *multiplicative* association through which an extra representation of the current state is integrated into the action value gradients to provide more information about the environment for the agents to address credit assignment implicitly through learning.

Concretely, let us first consider the partial derivative $\frac{\partial Q^\pi_\phi}{\partial \theta} = \frac{\partial Q^\pi_\phi}{\partial u}\frac{\partial u}{\partial \theta}$ for updating the policy parameters θ. When comparing the structures of the mixing critic (abbreviated as C_{Mix}) and the MLP critic (C_{MLP}), we only need to focus on $\partial Q^\pi_\phi / \partial u$ as $\partial u / \partial \theta$ is unrelated to the critic. Let us then consider the general case where both C_{MLP} and C_{Mix} operate on learned representations of s and u, denoted as $f_s(s)$ and $f_u(u)$ with non-linear f_s and f_u (e.g. MLP at input heads). In both cases, we can write:

$$\frac{\partial Q^\pi_\phi}{\partial u} = \frac{\partial Q^\pi_\phi}{\partial \nu}\frac{\partial \nu}{\partial u}, \text{ with } \nu = \begin{cases} f_s(s) + f_u(u) \\ f_s(s) f_u(u) \end{cases} \text{ and } \frac{\partial \nu}{\partial u} = \frac{\partial \nu}{\partial f_u}\frac{\partial f_u}{\partial u} = \begin{cases} \frac{\partial f_u}{\partial u}, & \text{for } \text{C}_{\text{MLP}}. \\ f_s(s)\frac{\partial f_u}{\partial u}, & \text{for } \text{C}_{\text{Mix}}. \end{cases} \quad (1)$$

Here, ν is the first *mixed* representation of $f_s(s)$ and $f_u(u)$ in the critic network before subsequent non-linearity: for C_{MLP}, it is the hidden representation immediately after the concatenation and the subsequent linear transform; for C_{Mix}, it is the representation immediately after the linear transform by the state-dependent hyper-weights $\mathbf{W}_1$ in Fig. 1 (b), which can be considered $f_s(s)$. As the rest of the critic network $g(\nu) = Q$ is learned, non-linear and non-interpretable in both cases, the key difference from Eq. 1 is thus that C_{Mix} changes the composition of $\partial Q^\pi_\phi / \partial \theta$ and introduces an extra, direct state representation $f_s(s)$ to facilitate policy learning towards implicit credit assignment.

In fact, similar techniques for altering gradient properties with multiplicative association were explored in the computer vision [9] and sequence modeling [54] literature, where element-wise multiplications between hidden features can act as a "gating mechanism" for information flow. In our case, while we do not focus on gating and the change in the gradient does not by itself guarantee a better credit assignment, we empirically show that the better utilization of the state made available from CTDE by C_{Mix} provides a better basis for learning credit assignment and better joint policies.

Critic Learning. We train Q^π_ϕ on-policy with a practical variant of generalized advantage estimation [41] and TD(λ) [47] adapted from [10, 39], with targets $y^{(\lambda)}_t$ defined recursively up to T, as:

$$\min_\phi \mathbb{E}_{\tau\sim\pi_\theta(\tau)}\left[\left(y^{(\lambda)}_t - Q^\pi_\phi(s_t, u_t)\right)^2\right], \quad y^{(\lambda)}_t = r_t + \gamma\left(\lambda y^{(\lambda)}_{t+1} + (1-\lambda) Q^\pi_\phi(s_{t+1}, u_{t+1})\right). \quad (2)$$

One could replace $y^{(\lambda)}_t$ with samples of R_t from reward trajectories as unbiased estimates of Q^π, though such estimates may have higher variance that scales with T. One may also use a *target* critic network $Q^\pi_{\phi^-}$ with periodic updates $\phi^- \leftarrow \phi$ to improve overall learning stability [31, 10, 39].

Policy Learning. With $Q^\pi_\phi \approx Q^\pi$, we train decentralized policies simultaneously to maximize the following objective, where $\mathcal{H}$ is the entropy regularization term for each agent (more in Section 3.3):

$$J(\theta) = \mathbb{E}_{\tau\sim\pi_\theta(\tau)}\left[Q^\pi_\phi(s_t, u_t) + \mathbb{E}_a\left[\mathcal{H}\left(\pi^a_{\theta_a}(\cdot|z^a_t)\right)\right]\right]. \quad (3)$$

In particular, we learn stochastic policies over deterministic policies because: (1) they can better handle state aliasing [14] and are harder to exploit [6] under partial observability; (2) there exists

POMDPs where stochastic policies can be arbitrarily better than any deterministic policies [44, 19, 21]; and (3) they admit on-policy learning and exploration [15, 14]. With small and discrete action spaces, we can compute the policy gradients from Eq. 3 directly with backpropagation as the expected Q over the actions can be obtained as $\pi_\theta(\cdot|z)^\top Q^\pi_\phi(s,\cdot)$ which is differentiable w.r.t. to θ. However, in the multi-agent case where the joint action space U^n grows exponentially, this is impractical and the expectation over the actions is estimated by sampling from the stochastic policies, which is not directly differentiable. Drawing connections to SVG(0) [15], one can backpropagate through samples by *reparameterizing* the stochastic policies as deterministic functions of independent noises; in the discrete case, this can be achieved using Gumbel-Softmax [20], a continuous, differentiable relaxation of the Gumbel-Max reparameterization for the categorical action samples. Here, we also consider an unconventional alternative previously explored in [52, 51] where we directly feed the *action distribution parameters* of each agent (e.g. mean and variance of a Gaussian policy and action probabilities of a discrete policy), denoted as $\pi^a_t := \pi^a_{\theta_a}(\cdot|z^a_t)$, as inputs to the critic for computing approximate policy gradients. Specifically, the policy of an agent a is updated using the following:

$$\frac{\partial J(\theta_a)}{\partial \theta_a} \approx \mathbb{E}_{\tau\sim\pi_\theta(\tau)}\left[\frac{\partial Q^\pi_\phi(s_t,\pi^1_t,...,\pi^n_t)}{\partial \pi^a_t}\frac{\partial \pi^a_t}{\partial \theta_a} + \frac{\partial \mathcal{H}\left(\pi^a_t\right)}{\partial \theta_a}\right]. \tag{4}$$

This formulation has several notable properties: during policy training, it can be considered a special case of DPG as the agents *deterministically* map their observations into the action parameters [14], allowing end-to-end differentiability without continuous approximations of action samples; the exploration of the action parameters is likewise influenced by entropy regularization, as is the previous case with sampling from the stochastic policies; and in contrast to the previous case, the action distributions contain more information (than sampled actions) about the agents' behaviors [52], which intuitively can improve the learning efficiency towards better joint actions. In the discrete case, the approximation in Eq. 4 induces a slight inconsistency between how the critic is used (probabilistic actions as inputs) and trained (special cases with probabilities 0 or 1); in practice, however, we show that policies can indeed converge (and often faster) to good solutions in complex scenarios.

3.3 Adaptive Entropy Regularization

As LICA is policy-based, it shares a common limitation that insufficient exploration may lead to premature convergence to poor policies. This is particularly notable when agents are acting in real-world environments where sampling a large number of diverse trajectories can be expensive or impractical. To this end, previous methods rely on an additional entropy term $\mathcal{H}\left(\pi^a(\cdot|z^a)\right) = \beta H\left(\pi^a(\cdot|z^a)\right) = \beta\mathbb{E}_{u^a\sim\pi^a}\left[-\log\pi^a(u^a|z^a)\right]$ for agent a weighted by some constant β during optimization. The rationale is that penalizing a low policy entropy resulting from under/over-confident actions should encourage sustained exploration throughout training. However, in practice we observe that this may not be the case: β often has high sensitivity in complex environments and a tiny nudge could lead to drastically different entropy trajectories; moreover, once policies start to converge (i.e. drop in entropy), the same regularization term may not encourage further exploration.

We hypothesize that these issues stem from the *curvature* of the entropy function derivative which in turn influences the magnitudes of its policy gradients on different action probabilities. Concretely, let us first consider the k-class action probability vector $p^a = \pi^a(\cdot|z^a)$ of an agent a with the number of feasible actions $k > 2$. The derivative of $\mathcal{H}$ with respect to p^a is given by:

$$d\mathcal{H} = \left[\frac{\partial\mathcal{H}}{\partial p^a_1},\cdots,\frac{\partial\mathcal{H}}{\partial p^a_k}\right] = [-\beta(\log p^a_1+1),\cdots,-\beta(\log p^a_k+1)] \tag{5}$$

assuming natural logarithm. We denote $d\mathcal{H}_i := -\beta(\log p^a_i + 1)$ and note that $d\mathcal{H}_i$ depends only on the i-th action probability p^a_i when $k > 2$. In particular, since $d\mathcal{H}_i$ is log-shaped, the gradient magnitudes for dampening high-confidence actions (large p^a_i) are disproportionately small compared to that for encouraging low-confidence actions (small p^a_i). While favoring the latter should help achieve the former, we argue that in practice, insufficient penalties for over-confidence can still result in agents sticking to a subset of high-confidence actions while only updating the action probabilities of other actions during stochastic optimization. On this basis, a reasonable approach is to manually induce a larger penalty (gradient magnitudes) for large p^a_i. To this end, we propose a practical technique called *adaptive entropy regularization* where we dynamically control the magnitudes of the entropy gradients such that they are inversely proportional to the policy entropy itself during training:

$$d\mathcal{H}_i := -\xi\cdot\frac{\log p^a_i + 1}{H(p^a)} \tag{6}$$

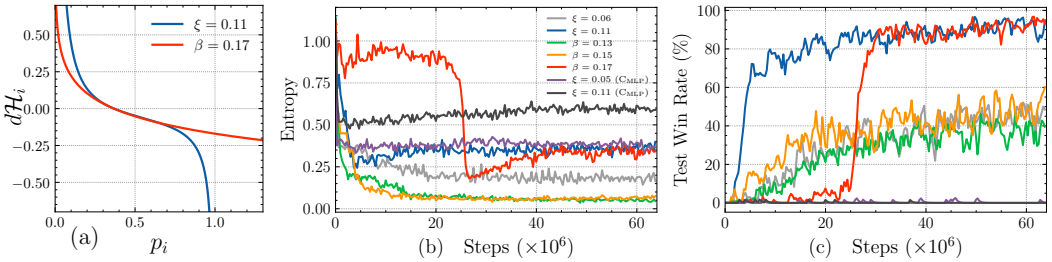

Figure 2: **Effects of *adaptive entropy regularization* and *mixing critic***. (a) Plots of the entropy function derivative before (with β) and after (with ξ) applying Eq. 6. (b-c) Entropy trajectories and test win rates on StarCraft II `5m_vs_6m`. Runs not labeled with C_{MLP} use a mixing critic (C_{Mix}).

with constant ξ controlling regularization strength. We visualize its values and curvature in Fig. 2 (a)[2], and note that large action probabilities now incur greater penalties from the entropy regularization.

This formulation has a clean interpretation that the regularization strength previously controlled by the constant β is now *adaptive* based on the policy stochasticity: if the policy exhibits deterministic behavior (i.e. low entropy), then the strength is scaled up by dividing a smaller term (the entropy); and if the policy is too stochastic, the strength is scaled down by dividing a larger term. In the multi-agent case, the entropy measure is defined per-agent, though in practice one could simplify and average it across all agents. While we do not provide a convergence analysis here, we show (Section 4.3.2, Fig. 2) that in practice, policies can indeed reach a stochasticity equilibrium (with entropy level determined by ξ), which can possibly help maintain a smooth objective landscape [1] and aid policy convergence. Possible future work may also generalize Eq. 6 by replacing $H(p^a)$ with other policy stochasticity measures, such as a multiplicative inverse of the L^2 norm of the difference between p_a and a uniform action probability vector, that may lead to different empirical or theoretical outcomes.

4 Experiments

4.1 Multi-Agent Particle Environments

We first evaluate our algorithm against previous state-of-the-art methods on two common multi-agent particle environments [27]: Predator-Prey and Cooperative Navigation.

Predator-Prey. In this environment, 3 cooperating agents control 3 predators to chase a faster prey by controlling their velocities with actions [`up`,`down`,`left`,`right`,`stop`] within an area containing 2 large obstacles at random locations. To focus on cooperative behavior, we follow [49] where the prey acts randomly. Each predator only observes its own velocity and position and its displacement from the prey, obstacles, and other predators. The goal is to capture the prey with a minimal number of steps, and the shared reward is the minimum distance between the prey and any of the predators. The game terminates when the prey is captured and an additional positive shared reward is given.

Cooperative Navigation. In this environment, n agents and n landmarks are initialized with random locations within an area, and the agents must cooperate to cover all landmarks by controlling their velocities with actions [`up`,`down`,`left`,`right`,`stop`]. Each agent only observes its own velocity and displacement from other agents and the landmarks, and the shared reward is the negative sum of displacements between each landmark and its nearest agent. Agents must also avoid collisions, as each agent incurs -1 shared reward for every collision against other agents. The environment uses $n = 3$ by default, and we also extend the difficulty with $n = 5$.

Training Settings. For all environments, agents are trained for 5000 episodes, each has a maximum of 200 steps and may end early for Predator-Prey. We follow the open-source implementations from [49] for the baseline algorithms, including COMA [10], independent DDPG [26] (IDDPG), MADDPG [27], and SQDDPG [49]. Each algorithm is trained with the same 5 random seeds and the mean and standard deviation of the goal metric (steps per episode, and mean reward over all timesteps and agents, respectively) throughout training are reported. See Supplementary for further settings.

[2]Note that $k = 2$ is a special case where the action probabilities are interdependent and Eq. 5 does not apply. For visualization purposes, Fig. 2 (a) follows Eq. 5, but note that our analysis applies to $k > 2$.